TBL 3 / EXERCÍCIO 1

Ativar o CORS

Instalação

No terminal, dentro do projeto, o pacote CORS é instalado com:

```
npm install cors
```

O ZIP já inclui o Express, o cors e suas dependências na pasta **node_modules**. Não é necessário instalar pacotes para executar os exercícios; basta ter o Node.js com npm instalado.

Servidor com CORS habilitado

```
const express = require('express');
const cors = require('cors');

const app = express();

app.use(cors());

app.get('/usuarios', (req, res) => {
  res.json([{ id: 1, nome: 'Ana' }]);
});

app.listen(3000, () => {
  console.log('Servidor rodando em http://localhost:3000');
});
```

O middleware é habilitado antes da rota /usuarios. A configuração **cors()** adiciona o cabeçalho Access-Control-Allow-Origin: *, permitindo a leitura da resposta por páginas de outras origens neste exemplo.

Resultado do teste

```
GET http://localhost:3000/usuarios
HTTP 200
Access-Control-Allow-Origin: *

[{ "id": 1, "nome": "Ana" }]
```

A chamada funcionou no navegador com o frontend nas portas 5500 e 5501.

Provocar e observar o erro

Teste sem CORS

No arquivo exercicio2/sem-cors.js, o backend foi executado sem app.use(cors()). O frontend permaneceu em http://localhost:5500, enquanto a API estava em http://localhost:3000.

```
npm run ex2-sem-cors
```

Ao recarregar a página, o navegador bloqueou a leitura da resposta. Trecho da mensagem registrada no console:

```
Access to fetch at 'http://localhost:3000/usuarios'
from origin 'http://localhost:5500' has been blocked
by CORS policy: No 'Access-Control-Allow-Origin' header
is present on the requested resource.
```

Por que funciona no Postman e falha no navegador?

O Postman faz requisições HTTP sem aplicar a política de mesma origem dos navegadores. Por isso, ele consegue mostrar a resposta da API mesmo quando ela não contém os cabeçalhos de CORS necessários para o frontend.

O navegador verifica esses cabeçalhos antes de liberar a resposta para o JavaScript. Como as portas 5500 e 3000 formam origens diferentes, a ausência de permissão impede a leitura. A API pode receber a requisição e responder HTTP 200, mesmo com esse bloqueio no navegador.

CORS habilitado novamente

```
const cors = require('cors');
app.use(cors());
```

Depois de encerrar a versão sem CORS, foi executado o arquivo exercicio2/com-cors.js:

```
npm run ex2-com-cors
```

Ao recarregar o frontend, a consulta voltou a funcionar e retornou o usuário Ana. A mensagem completa do erro está em resultados/erro_exercicio2.txt.

TBL 3 / EXERCÍCIO 3

Restringir a origem permitida

Configuração utilizada

```
app.use(cors({
  origin: 'http://localhost:5500'
}));
```

Essa configuração autoriza o navegador a liberar a resposta para scripts executados em http://localhost:5500. A origem precisa coincidir em protocolo, host e porta.

Resultado dos testes

Origem do frontend	Resultado no navegador
http://localhost:5500	Leitura permitida. Retorno do usuário Ana.
http://localhost:5501	Leitura bloqueada por CORS.

O que acontece em uma origem diferente?

Se a página for servida em http://localhost:5501, por exemplo, sua origem será diferente da origem autorizada. O backend continua respondendo, mas o navegador não entrega a resposta ao JavaScript daquela página, e o fetch termina com erro.

Um endereço como http://127.0.0.1:5500 também representa outra origem, porque o host é diferente de localhost.

Como reproduzir

```
npm run ex3

npm run frontend

npm run frontend-outra-origem
```

Execute cada comando em um terminal separado. Acesse as portas 5500 e 5501 e compare o console. O registro do bloqueio está em resultados/erro_exercicio3.txt.

TBL 3 / EXERCÍCIO 4

Testar a API com um frontend

HTML utilizado

```
<!DOCTYPE html>
<html lang="pt-BR">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Consulta de usuários</title>
</head>
<body>
  <h1>Usuários</h1>
  <p>O resultado também aparece no console do navegador (F12).</p>
  <pre id="resultado">Consultando a API...</pre>

  <script>
    fetch('http://localhost:3000/usuarios')
      .then(res => res.json())
      .then(dados => {
        console.log(dados);
        document.getElementById('resultado').textContent =
          JSON.stringify(dados, null, 2);
      })
      .catch(erro => {
        console.error(erro);
        document.getElementById('resultado').textContent =
          'Não foi possível consultar a API.';
      });
  </script>
</body>
</html>
```

Resultado exibido no console

```
[
  { "id": 1, "nome": "Ana" }
]
```

O console recebeu um array com um objeto. O conteúdo foi expandido acima para facilitar a leitura. A rota GET /usuarios respondeu HTTP 200 e o mesmo resultado apareceu na página.

Execução: npm run ex4 e, em outro terminal, npm run frontend. Página: http://localhost:5500. Registro: resultados/console_exercicio4.txt.

Permitir apenas o método GET

Configuração utilizada

```
app.use(cors({
  origin: 'http://localhost:5500',
  methods: ['GET']
}));
```

Objetivo de limitar os métodos

A intenção é permitir a consulta de dados pelo frontend e restringir métodos de alteração na política de CORS. A opção methods configura o cabeçalho Access-Control-Allow-Methods enviado nas respostas ao preflight.

No teste, o GET retornou os usuários normalmente. Uma tentativa de DELETE gerou um preflight e foi bloqueada pelo navegador, pois esse método não constava nos métodos informados pelo servidor.

Resposta ao preflight do teste

```
HTTP 204
Access-Control-Allow-Origin: http://localhost:5500
Access-Control-Allow-Methods: GET
```

Limite dessa configuração

A configuração de CORS não é um bloqueio geral de métodos no servidor. GET, HEAD e POST são métodos considerados simples pelo padrão Fetch; quando a requisição atende às condições de uma requisição simples, não há preflight. Portanto, methods: ['GET'] não basta para impedir todas as tentativas de escrita.

Nesta API, foi implementada somente a rota GET /usuarios. Um POST simples retornou HTTP 404 por não existir rota POST. O controle efetivo das operações deve ser feito nas rotas e nas regras de autorização do backend.

Referências

Material de apoio: materialTBL3(1).pdf. Atividade: exTBL3(1).pdf.

Documentação oficial do middleware cors: expressjs.com/en/resources/middleware/cors/

Padrão Fetch, preflight: fetch.spec.whatwg.org/#cors-preflight-fetch